

Q Given an array, find the total sum of all possible subarrays.

Ex. arr[] = [1 2 3]

[1] $\rightarrow$ 1

[1 2] $\rightarrow$ 3

[1 2 3] $\rightarrow$ 6

[2] $\rightarrow$ 2

[2 3] $\rightarrow$ 5

[3] $\rightarrow$ 3

20

int subarraySum (int arr[], int n) {

int sum = 0

for (int i = 0; i < n; i++) { $\rightarrow$ fixing start point

for (int j = i; j < n; j++) { $\rightarrow$ fixing end point

for (int k = i; k <= j; k++)

sum = sum + arr[k];

}

}

return sum;

}

Tc: $O(n^3)$

→ Using prefix sum array.

```
int subarraySum (int arr[], int n) {
```

```
    int sum = 0
```

```
    for (int i = 0; i < n; i++) {
```

→ Fixing start point

```
        for (int j = i; j < n; j++) {
```

→ Fixing end point

```
            if (i == 0)
```

```
                sum = sum + arr[j];
```

```
            else
```

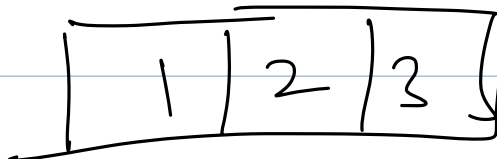
```
                sum = sum + arr[j] - arr[i-1];
```

```
        }
```

```
    return sum;
```

```
}
```

TC: $O(n^2)$
SC: $O(1)$



→ Without prefix sum

```
int subarraySum (int arr[], int n) {
```

```
    int total-sum = 0
```

```
    for (int i = 0; i < n; i++) {
```

```
        int subarray-sum = 0;
```

```
        for (int j = i; j < n; j++) {
```

```
            subarray-sum += arr[j];
```

```
            total-sum += subarray-sum;
```

```
    }
```

```
    return total-sum;
```

```
}
```



$[1 \ 2 \ 3]$
subarray-sum = 3
total-sum = 20

$$\text{arr } 1 = [1 \ 2 \ 3]$$

$$[1] \rightarrow 1$$

$$[1 \ 2] \rightarrow 3$$

$$[1 \ 2 \ 3] \rightarrow 6$$

$$[2] \rightarrow 2$$

$$[2 \ 3] \rightarrow 5$$

$$[3] \rightarrow 3$$

$$\underline{\underline{20}}$$

$$3 \times 1 + 4 \times 2 + 3 \times 3$$

$$\Rightarrow 3 + 8 + 9 = 20$$

In how many subarrays, the element at index 1 will be present?

A: [3, -2, 4, -1, 2, 6]

Possible Starting - points : $[0, 1] = 2$

Possible Ending - points : $[1, 5] = 5$

$$2 \times 5 = 10$$

Quiz 2

0 1 2 3 4 5
[3 -2 4 -1 2 6]

$[0, i]$ Possible Starting - points : $[0, 2] = 3$

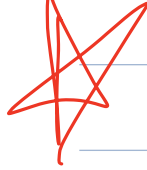
$[i, n-1]$ Possible Ending - points : $[2, 5] = 4$

$$3 \times 4 = 12$$

Possible Starting - points : $[0, i] \Rightarrow i - 0 + 1 = (i+1)$

Possible Ending - points : $[i, n-1] \Rightarrow n - 1 - i + 1 = (n-i)$

Possible Subarrays $\Rightarrow (i+1) \times (n-i)$
which contain i



```
int total-sum = 0
```

```
for (int i = 0 ; i < n ; i++) {
```

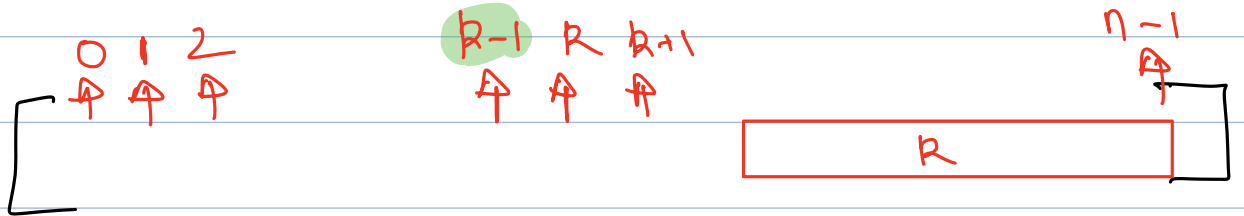
```
    total-sum += (arr[i]) * (i+1) * (n-i)
```

```
return total-sum;
```

$T_c: O(n)$, $Sc: O(1)$

Q Count subarrays of length k , given an array of length n .

$\Rightarrow n, k$



Possible End points : $[k-1, n-1]$



$$n - (k-1) + 1$$

$$\underline{n - k + 1}$$

$$n = 7, k = 4$$

$$= 7 - 4 + 1 \Rightarrow 4.$$

$$n = 7, k = 4$$

$a_0 \ a_1 \ a_2 \ a_3 \ a_4 \ a_5 \ a_6$

The subarray a_3, a_4, a_5, a_6 is highlighted with a purple box.

Q Given an array of n elements, find maximum subarray sum of length k .

$\Rightarrow n=8, k=4$

0	1	2	3	4	5	6	7
-3	4	-2	5	3	-2	8	2

s	e	Sum
0	3	4
1	4	10
2	5	4
3	6	14
4	7	11

0	1	2	3	(i)	5	6	(j)
-3	4	-2	5	3	-2	8	2

Subarray-sum $\Rightarrow 14 + 2 - 5 = 11$
max-sum $\Rightarrow$ ~~10~~ 14

```
int Subarray_sum = 0
```

```
for (int i=0; i < k; i++)  
    Subarray_sum += arr[i].
```

k

```
int max_sum = Subarray_sum
```

```
int i=0, j=k
```

```
while (j < n) {
```

```
    Subarray_sum = Subarray_sum  
    + arr[j] - arr[i].
```

```
    i++; j++;
```

```
    if (Subarray_sum > max_sum)  
        max_sum = Subarray_sum
```

```
}
```

```
return max_sum;
```

$n-k$

$TC: O(n)$

$SC: O(1)$

